



# **EC-220 Computer System Architecture**

---

**Dr. Ahsan Shahzad**

**Department of Computer and Software Engineering (DC&SE)**

**College of Electrical and Mechanical Engineering (CEME)**

**National University of Science and Technology (NUST)**

Week # 13\_OLE

# Overview of Pipelining

---

- **Pipelining**

- An implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.

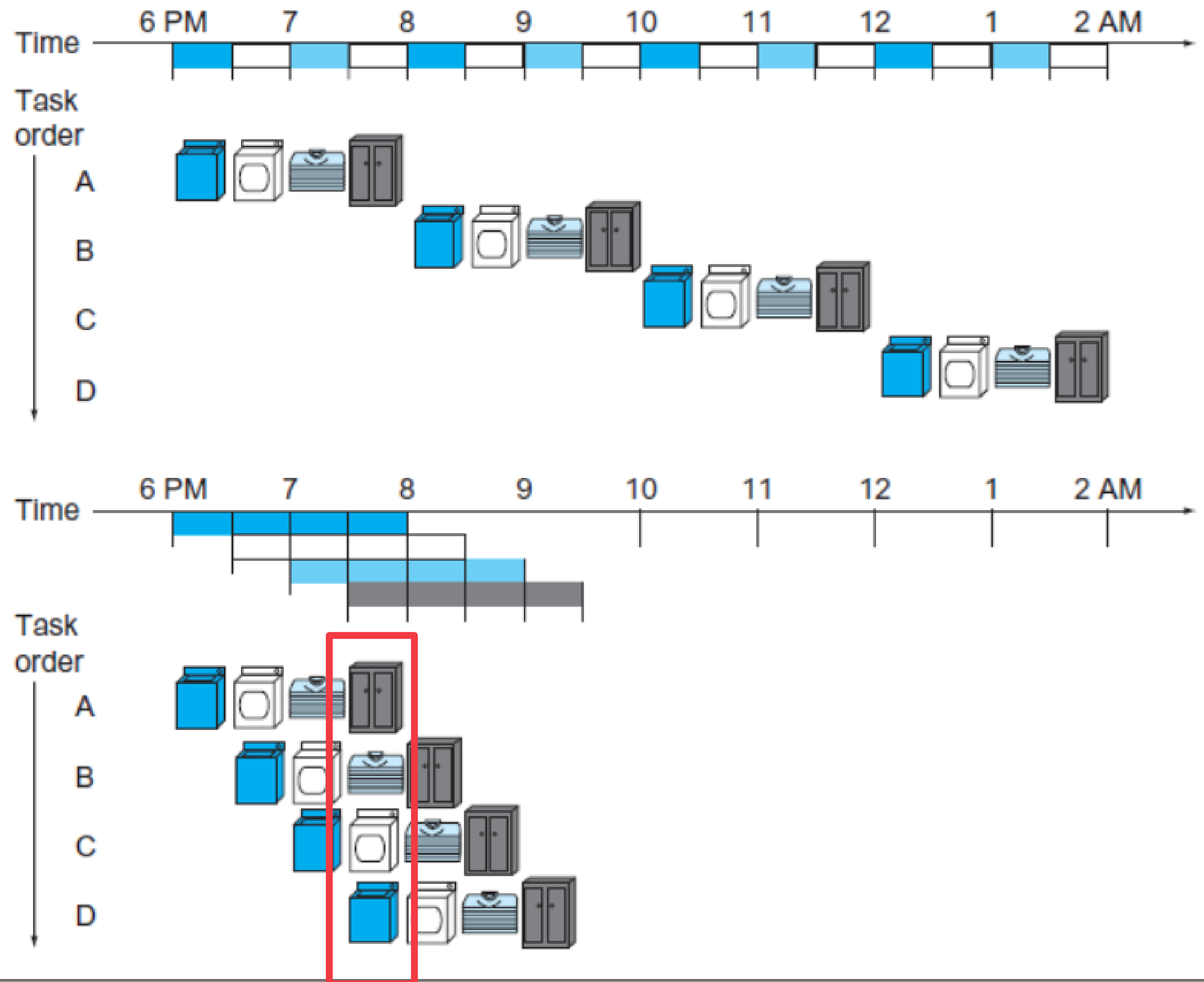
- **Analogy: Laundry Process**

- The nonpipelined approach to laundry would be as follows:
  1. Place one dirty load of clothes in the **washer**.
  2. When the washer is finished, place the wet load in the **dryer**.
  3. When the dryer is finished, place the dry load on a table and **fold**.
  4. When folding is finished, ask your roommate to **put** the clothes **away**.
- The pipelined approach to laundry would be as follows:
  - As soon as one resource got free, we will start using it for next round of clothes.
  - Pipelined approach takes much less time.



# Pipelining **Analogy**

- Individual steps in pipeline are called *stages*
- Will pipelining improves the time taken by each step/resource in laundry system?
- NO !!
- Pipelining improves **throughput**
  - pipelining is faster for many loads is that everything is working in parallel, so more loads are finished per hour



**FIGURE 4.25 The laundry analogy for pipelining.** Ann, Brian, Cathy, and Don each have dirty clothes to be washed, dried, folded, and put away. The washer, dryer, “folder,” and “storer” each take 30 minutes for their task. Sequential laundry takes 8 hours for 4 loads of wash, while pipelined laundry takes just 3.5 hours. We show the pipeline stage of different loads over time by showing copies of the four resources on this two-dimensional time line, but we really have just one of each resource.



# Performance Enhancement via Pipelining

## ■ Performance Improvement by Pipelining

- (Ideally) speed-up due to pipelining is equal to *no. of stages in the pipeline*
- If all stages take same amount of time & there is enough work to do.
- (Practically) this will give us upper bound of performance as stages normally don't exactly equal and there is some overhead costs associated with pipelining (*discussed in next slides*)

## ■ Analogy: Laundry Process

- According to above criteria:
  - Pipelined laundry can potentially be 4 times faster than nonpipelined
  - i.e. 20 loads will finish in 5 times the time for 1 load Versus 20 times the time for 1 load (nonpipeline)
  - In previous fig. 2.3 times ( $8 / 3.5$ ) for 4 loads
  - During start and end of workload: resources are not fully utilized
- If we have 1 load how much performance difference we get?

# Pipelining in Processors

---

- The same principles apply to **processors** where we **pipeline instruction-execution**.
- **MIPS IS** was designed for pipelining
- MIPS instructions classically take **five steps**:
  1. Fetch instruction from memory.
  2. Read registers while decoding the instruction. The regular format of MIPS instructions allows reading and decoding to occur simultaneously.
  3. Execute the operation or calculate an address.
  4. Access an operand in data memory.
  5. Write the result into a register.
- MIPS pipeline will have **five stages**
- Discussion will be limited to these **8 instructions**: lw, sw, add, sub, and, or, slt, beq



# Single Cycle Vs. Pipeline Processor Performance

- (Assume) Operational times for major functional units 200ps except Registerfile - 100ps
- Clock Cycle Time:
  - Single-Cycle Processor: 800ps
  - Pipeline Processor: 200ps

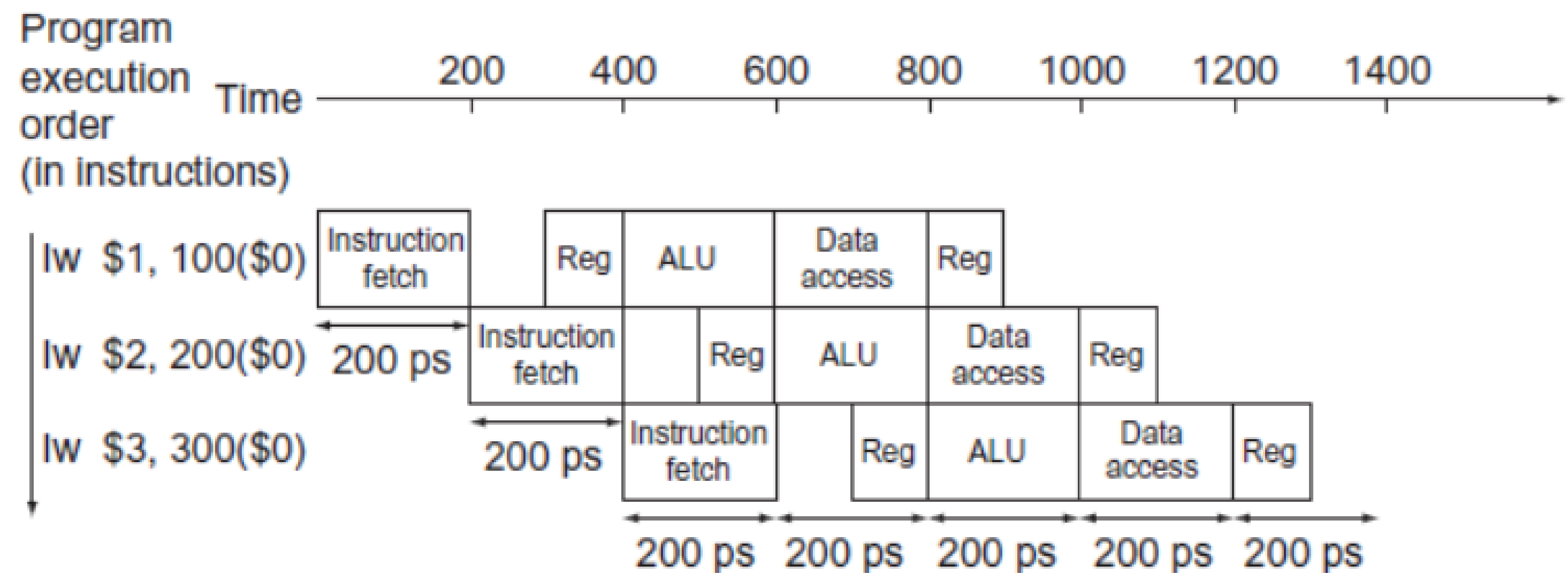
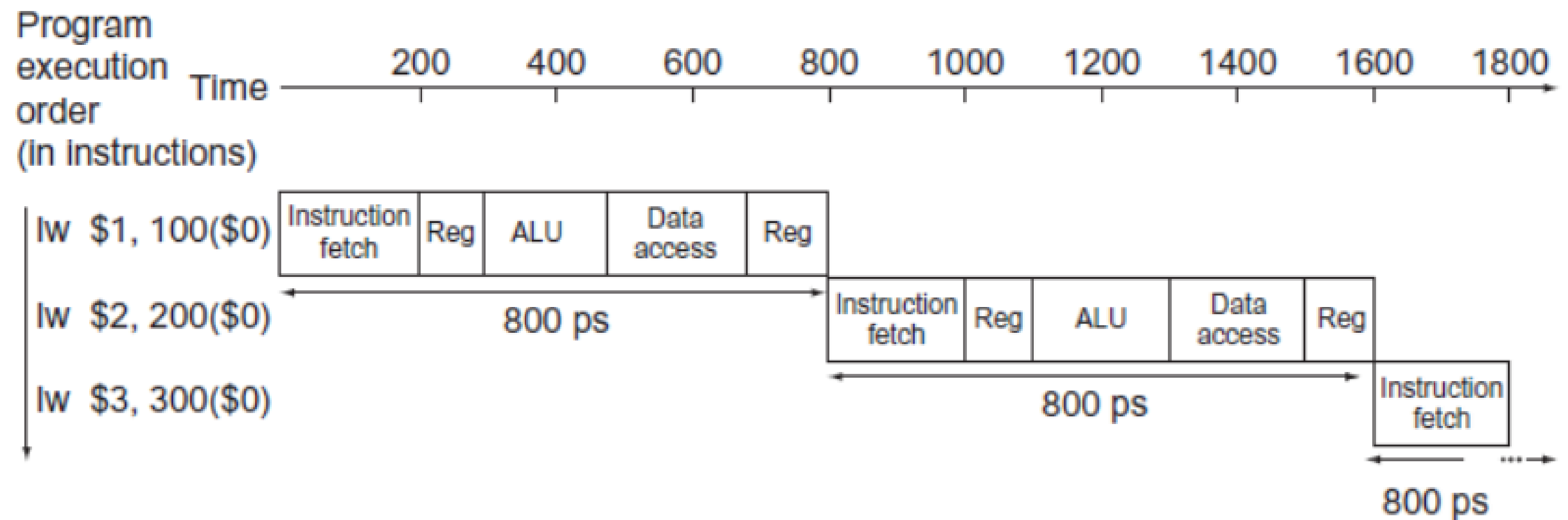
| Instruction class                 | Instruction fetch | Register read | ALU operation | Data access | Register write | Total time |
|-----------------------------------|-------------------|---------------|---------------|-------------|----------------|------------|
| Load word (lw)                    | 200 ps            | 100 ps        | 200 ps        | 200 ps      | 100 ps         | 800 ps     |
| Store word (sw)                   | 200 ps            | 100 ps        | 200 ps        | 200 ps      |                | 700 ps     |
| R-format (add, sub, AND, OR, slt) | 200 ps            | 100 ps        | 200 ps        |             | 100 ps         | 600 ps     |
| Branch (beq)                      | 200 ps            | 100 ps        | 200 ps        |             |                | 500 ps     |

**FIGURE 4.26** Total time for each instruction calculated from the time for each component. This calculation assumes that the multiplexors, control unit, PC accesses, and sign extension unit have no delay.

# (ctd.) Performance

- For a work load of 3 instrs.
- Time b/w start of 1<sup>st</sup> & 4<sup>th</sup> instr.
  - $3 \times 800\text{ps} = 2400\text{ps}$  (SC)
  - $3 \times 200\text{ps} = 600\text{ps}$  (Pipeline)
  - 4-fold speed up

- Pipeline stage times also limited by slowest resource
- Assuming Register File.
  - *Write* – first half of clk cycle
  - *Read* – 2<sup>nd</sup> half of clk cycle



**FIGURE 4.27 Single-cycle, nonpipelined execution in top versus pipelined execution in bottom.**

## (ctd.) Performance

- If the **stages are perfectly balanced**, then the time between instructions on the pipelined **processor**—**assuming ideal conditions**—is equal to

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of pipe stages}}$$

- Under ideal conditions, a five-stage pipeline will be 5 times faster.
- According to this,
  - In prev. example, nonpipeline was **800ps** ->  $800/5$  -> **160ps** should be clk cycle time of pipeline
  - Stages not balanced and there are some overheads involved with pipelining
- Performance Improvement in terms of **total execution time**
  - 1400ps Vs 2400 ps
  - Add 1 million (1,000,000) instrs. in the workload. Now ??

$$\frac{800,002,400 \text{ ps}}{200,001,400 \text{ ps}} \approx \frac{800 \text{ ps}}{200 \text{ ps}} \approx 4.00$$



# Designing Instructions Set for Pipelining

---

## **MIPS was designed for pipeline execution**

- All MIPS instrs. are having same length. Easy to fetch and decode
- Few instructions formats, Source register always at the same place. This symmetry facilitates instr. Decoding in parallel to register file read at the 2<sup>nd</sup> stage.
  - Otherwise, stage 2 needs to be split. Longer pipelines have their own downsides
- Memory operands only appear in loads and stores instrs. in MIPS.
  - i.e. we can calculate address at execute stage and then access memory
- Operands must be aligned in memory

# Pipelined Processor **Datapath**

---

- MIPS instr. Execution is divided into 5 steps
  - *5 stages in pipeline*
  - *5 instructions will be in execution during any single clock cycle.*
  - *Datapath must be splitted into 5 pieces*
- Each stage of datapath is named based on the instr. Execution step:
  - **IF:** Instruction fetch
  - **ID:** Instruction decode and register file read
  - **EX:** Execution or address calculation
  - **MEM:** Data memory access
  - **WB:** Write back

# Pipelined Datapath

IF: Instruction fetch

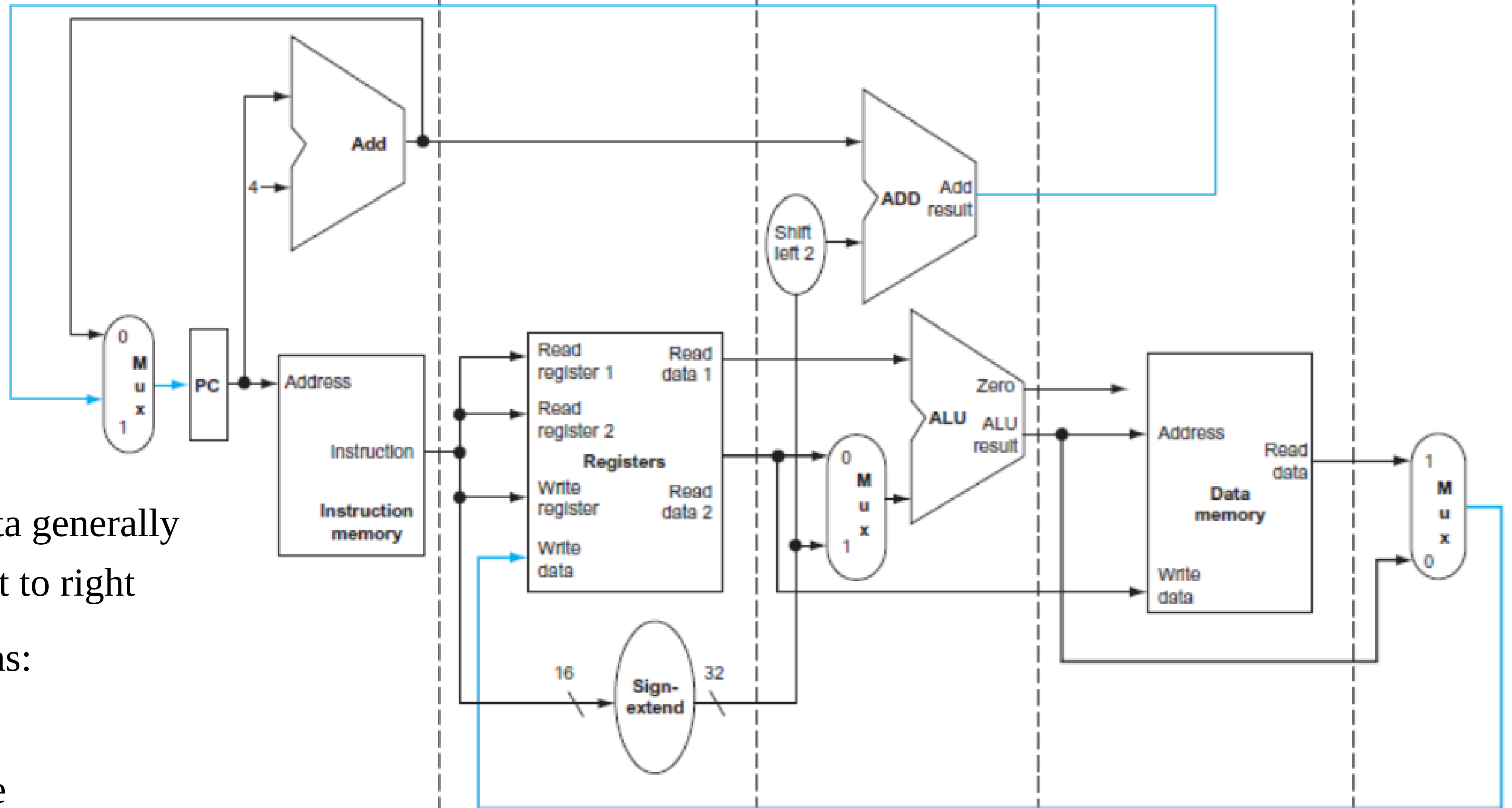
ID: Instruction decode/  
register file read

EX: Execute/  
address calculation

MEM: Memory access

WB: Write back

- Instrs. And data generally move from left to right
- Two exceptions:
  - WB step
  - PC update

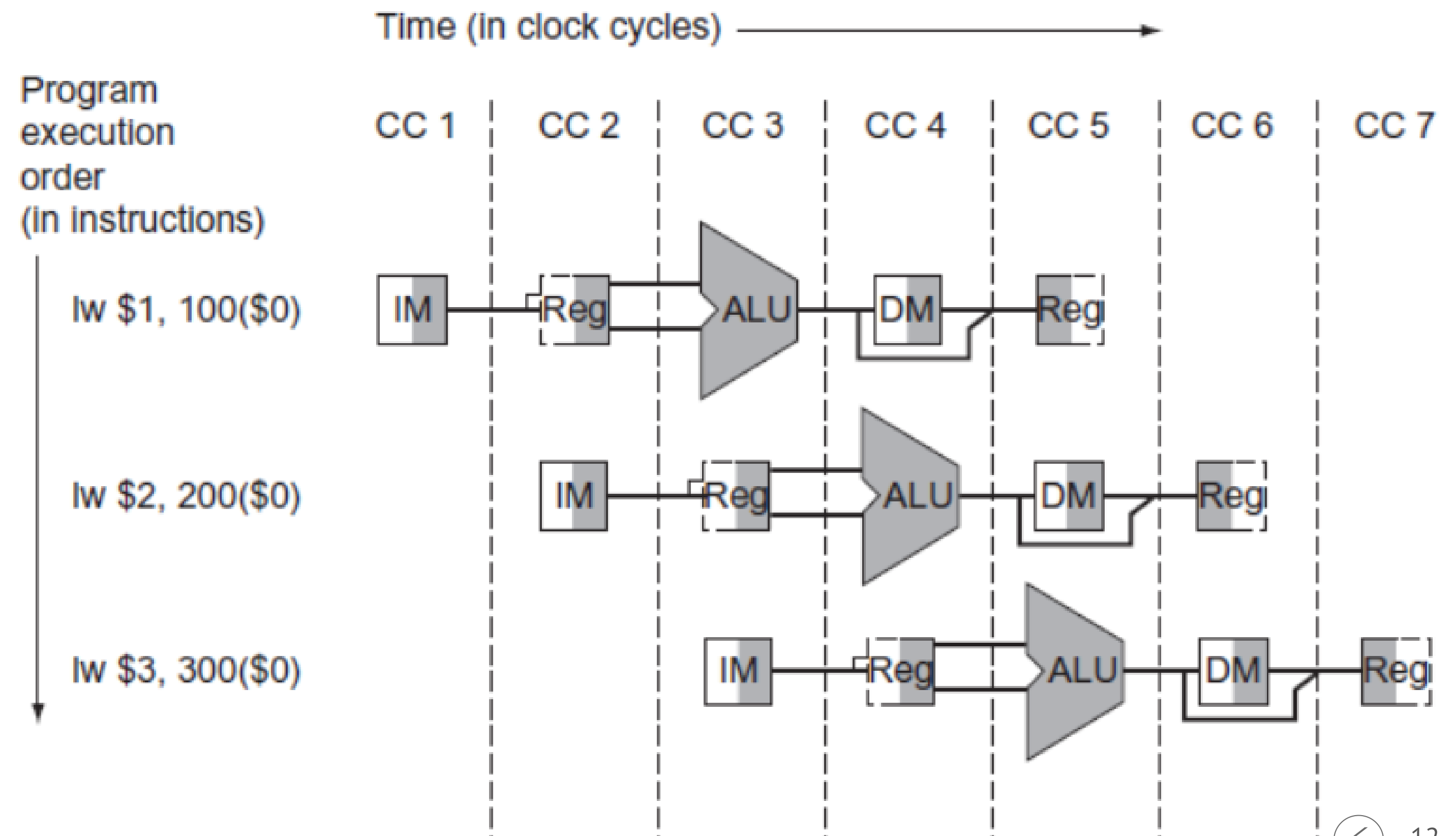




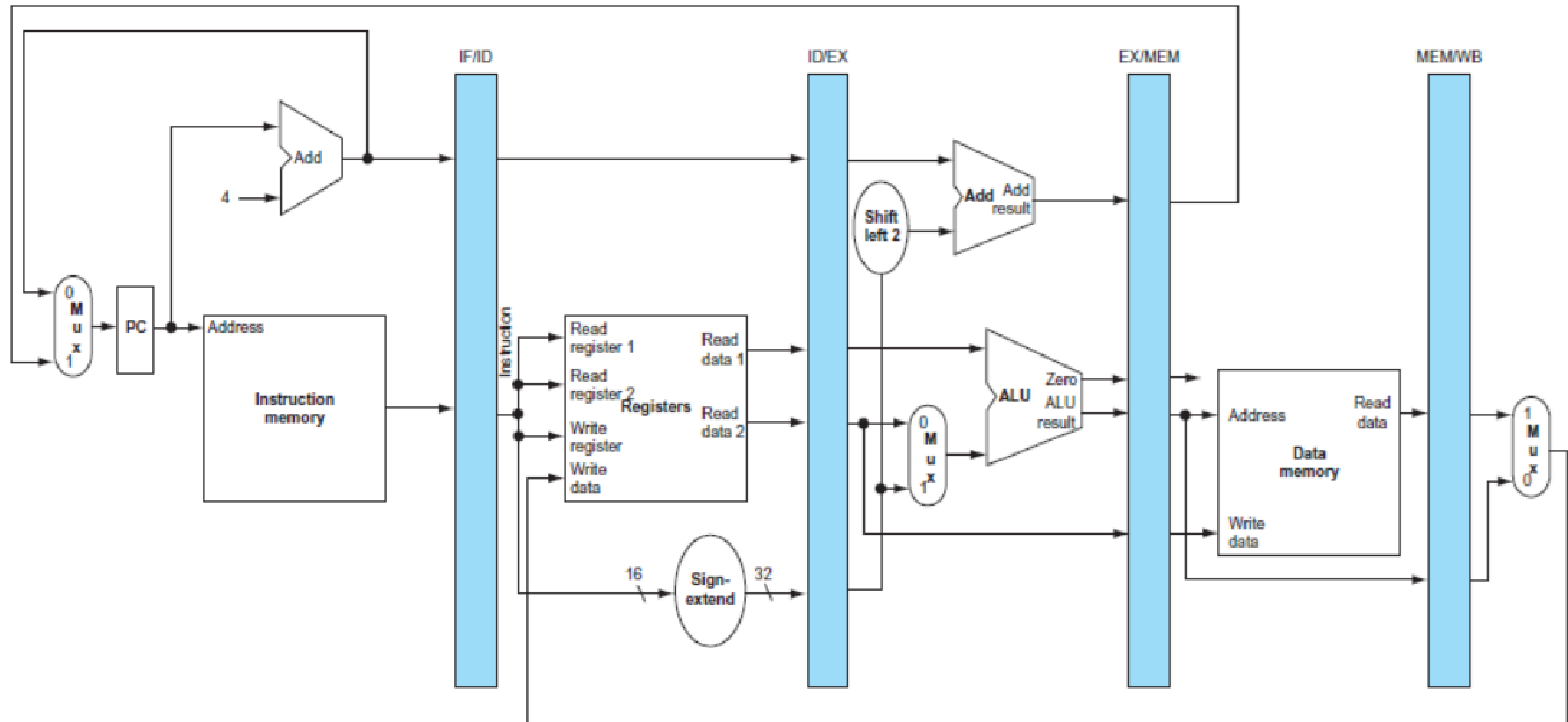
# Instructions Execution in **Pipelined Datapath**

- One representation: Multiple-Clock cycle diagram
- Fig. seems to suggest for 3 instructions we need 3 datapath
- Instead we shared the datapath by adding Pipeline Registers to hold intermediate data on instrs.

- E.g. Shared Instr. Memory
  - Instr. Fetch needs to be preserved for later stages
- Laundry Analogy
  - A basket b/w each pair of stages



# Pipelined Datapath



**FIGURE 4.35** The pipelined version of the datapath in Figure 4.33. The pipeline registers, in color, separate each pipeline stage.

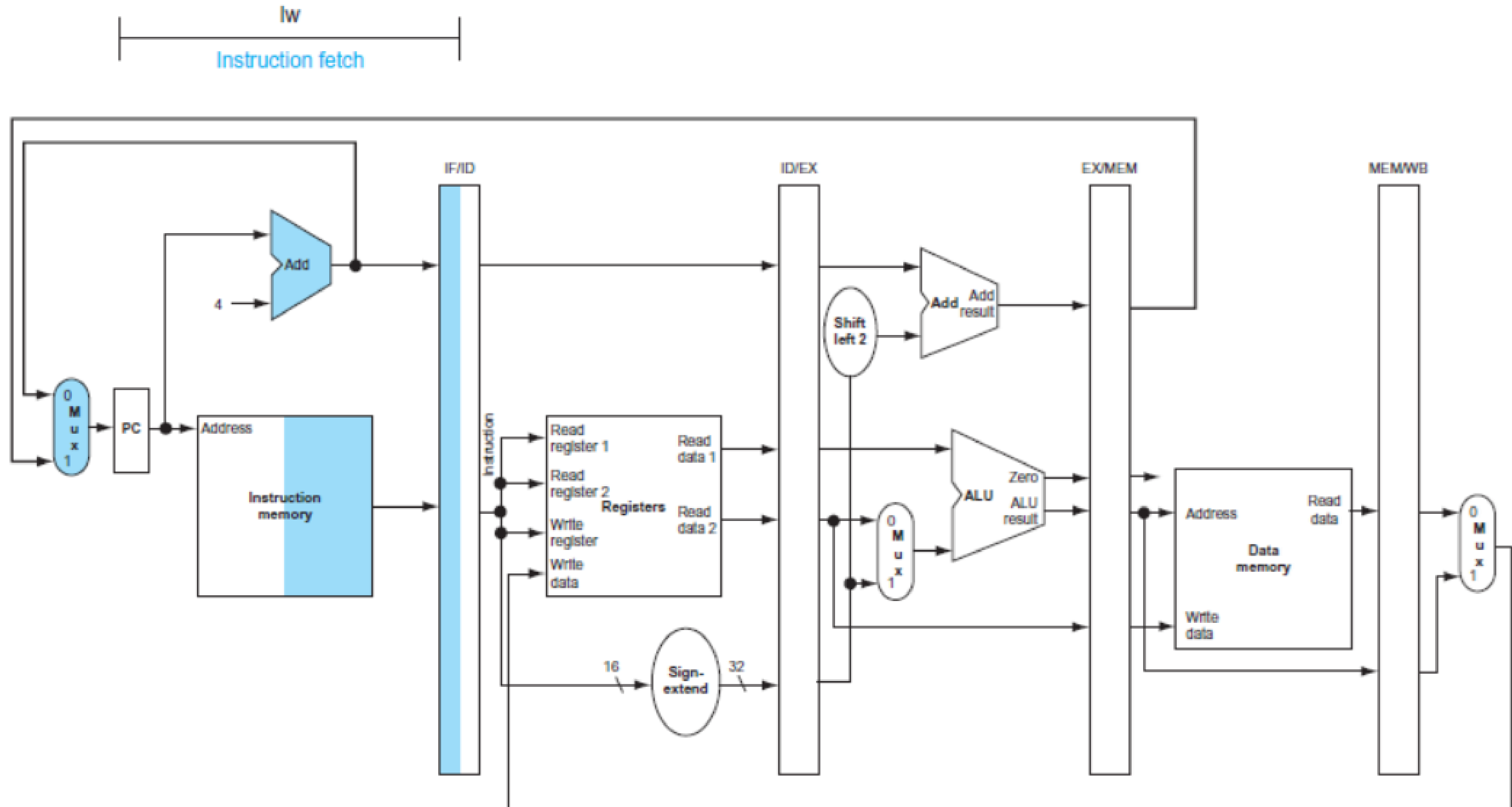


# Working Demonstration of **Pipelined Datapath**

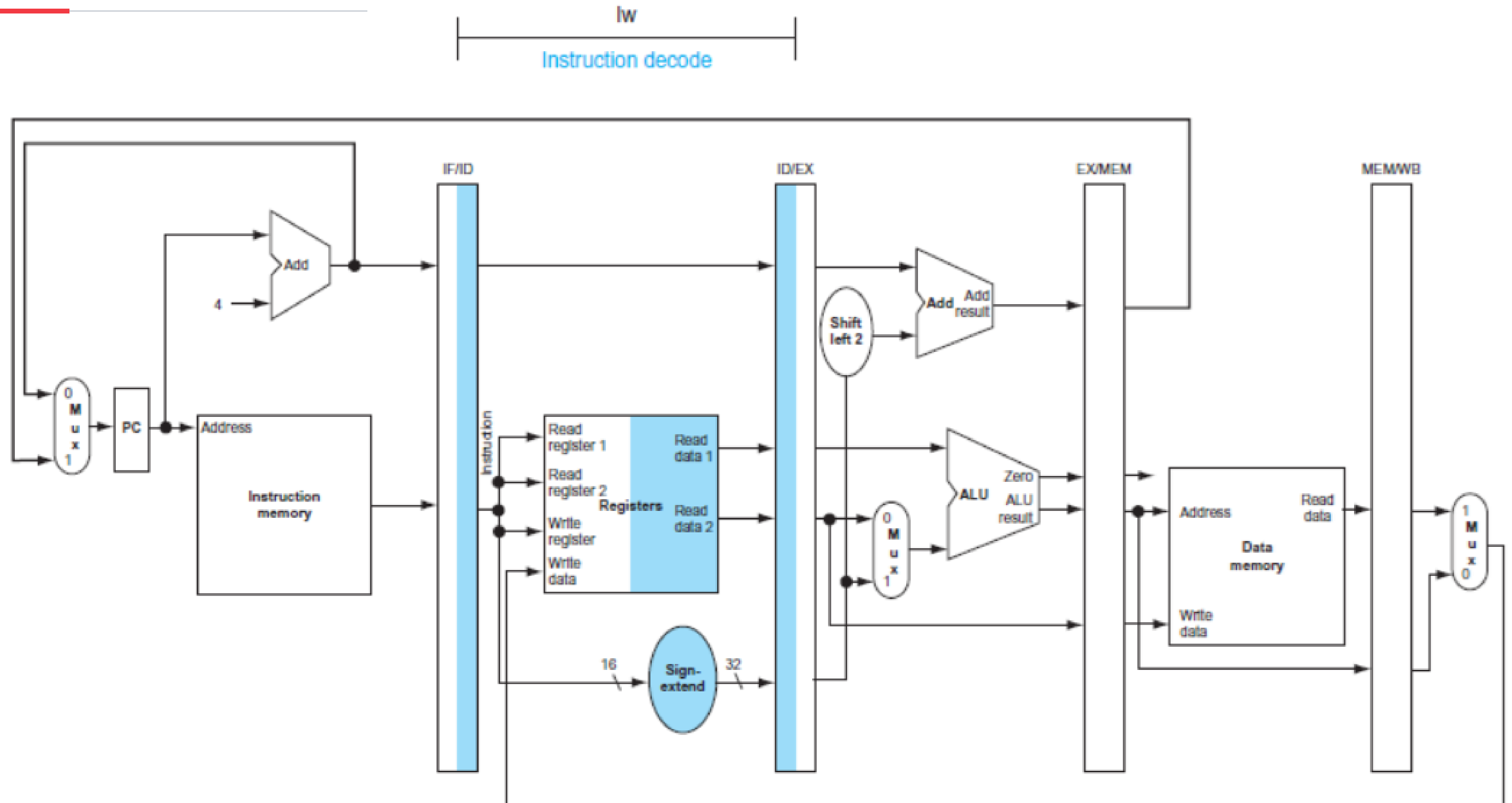
---

- To show how the pipelining works, Let's see sequences of figures to demonstrate operation over time.
- First we will see lw instruction as an example as it uses all the stages
- The active portions will be highlighted in upcoming pictures in next couple of slides
- Highlight Right half of Registers or Mem when they are read, left half when write.
- Processor must transfer everything to next Pipeline registers that might be needed by any instruction during a later clock cycle.

# Lw Example - Pipelined Datapath

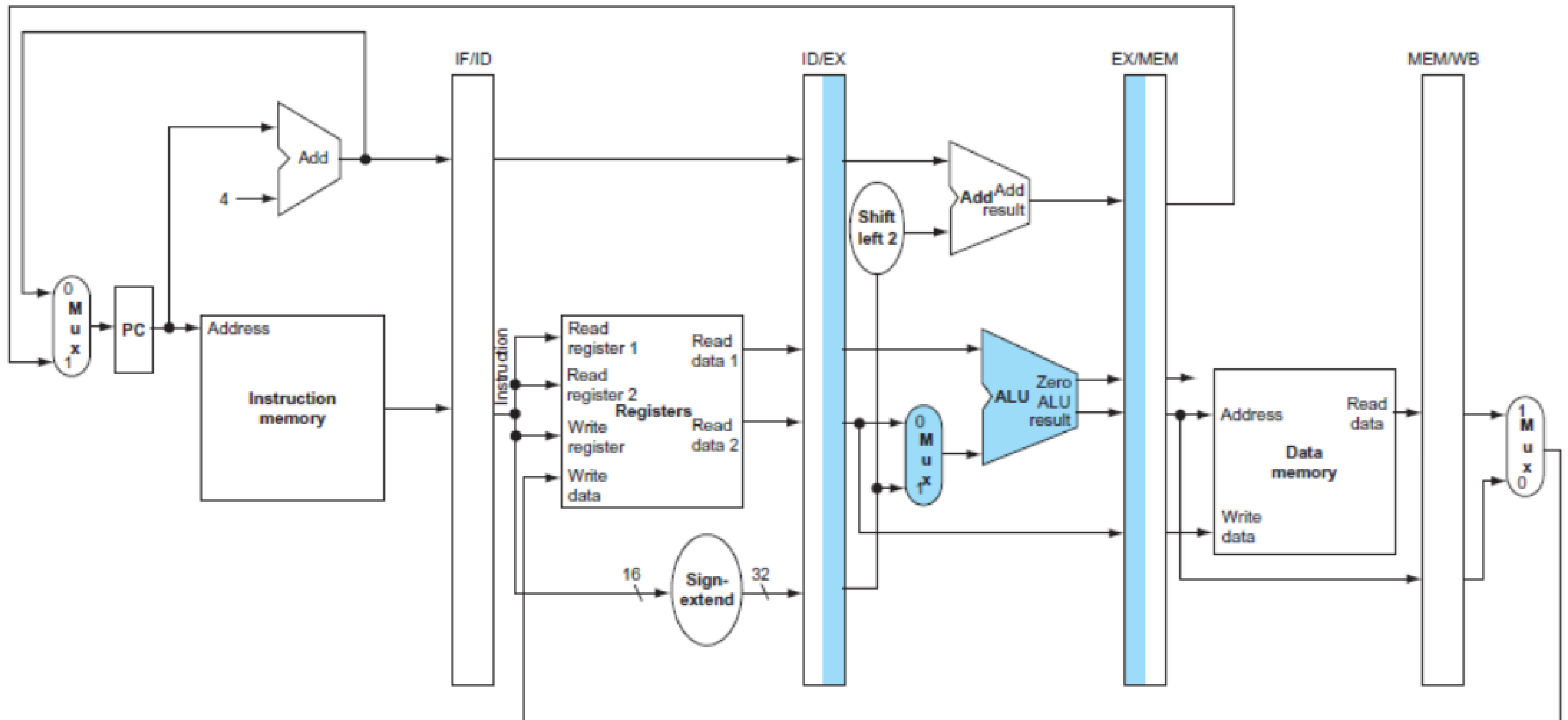


# Lw Example - Pipelined Datapath

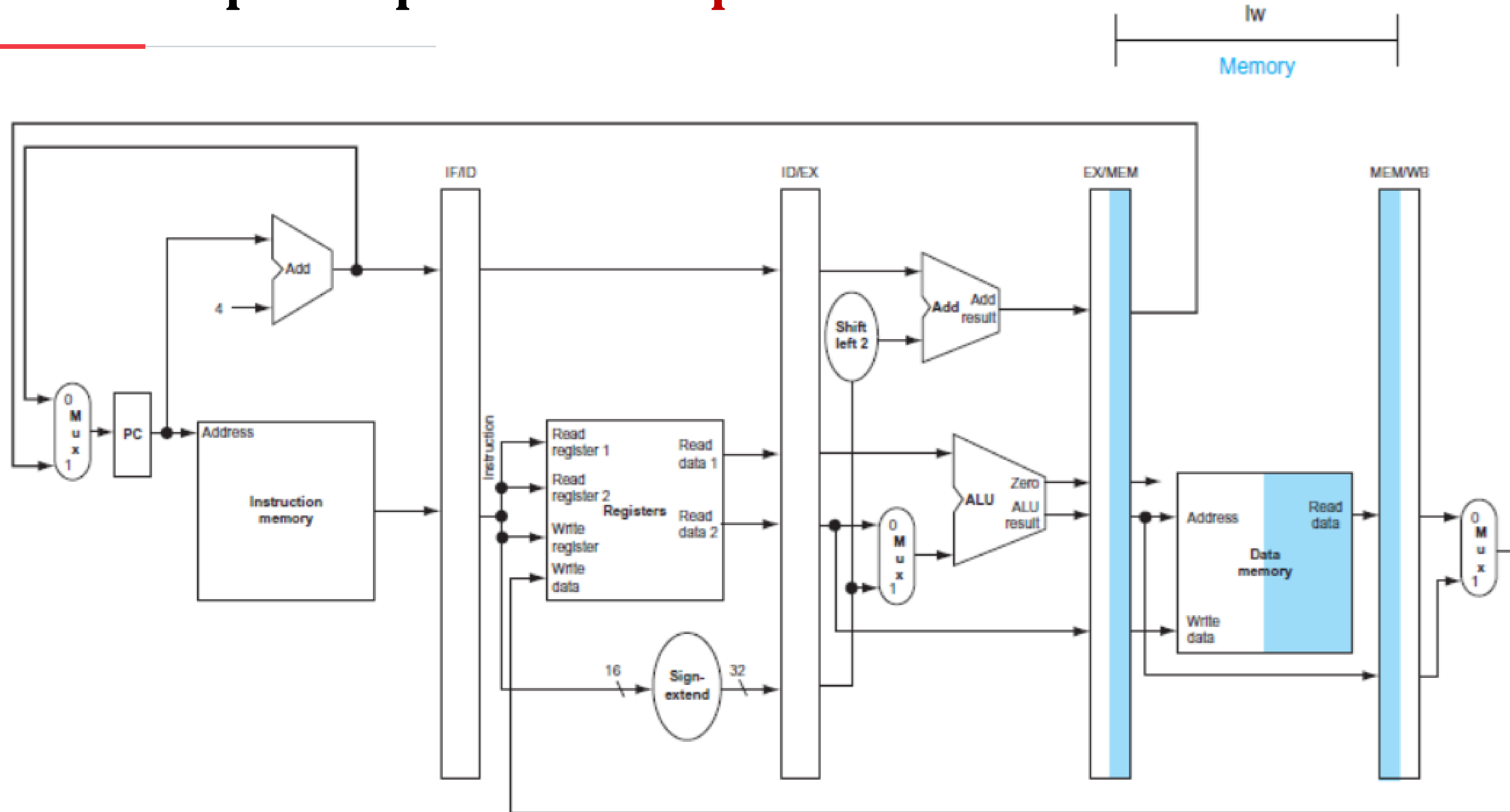




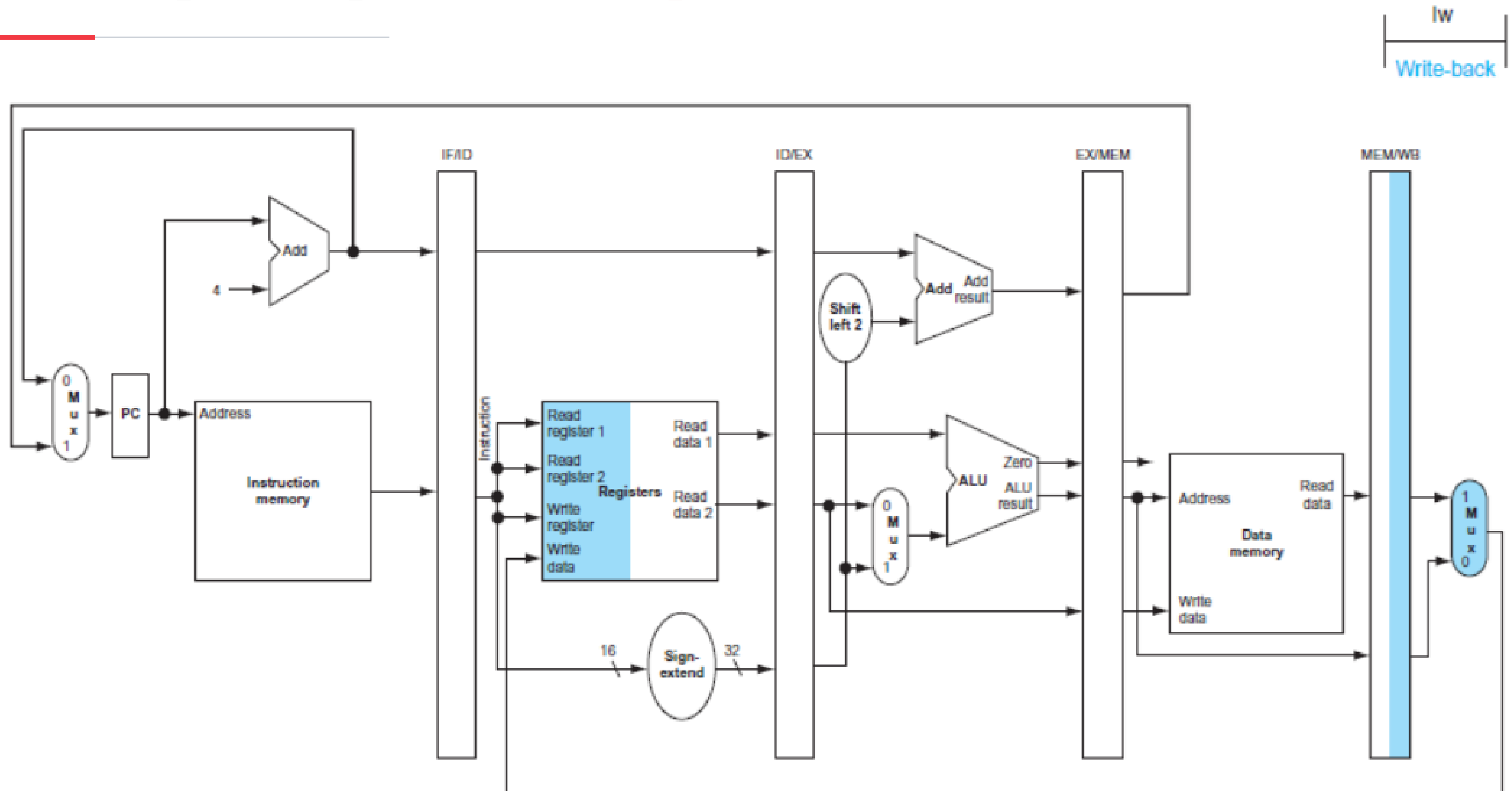
# Lw Example - Pipelined Datapath



# Lw Example - Pipelined Datapath

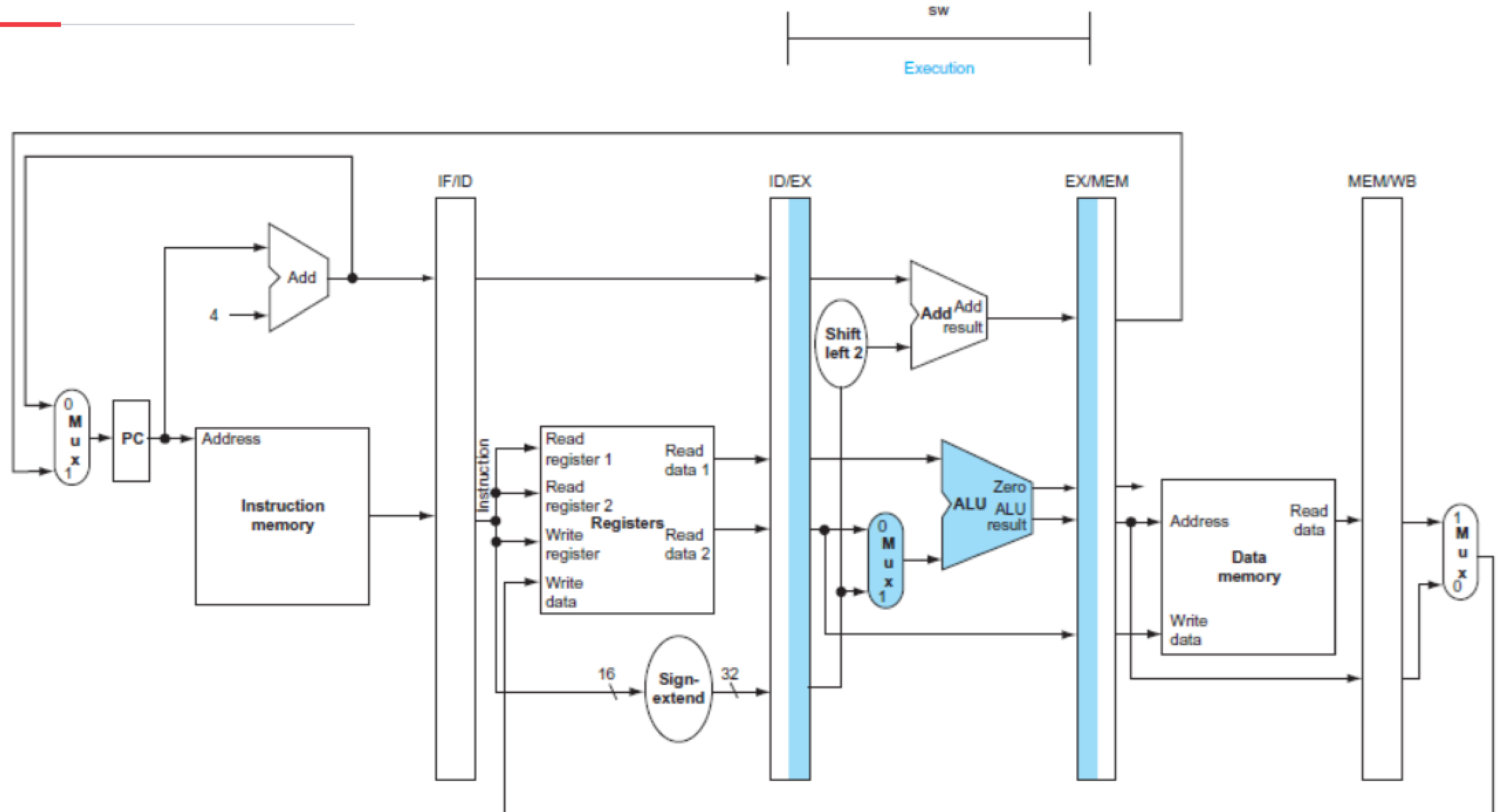


# Lw Example - Pipelined Datapath

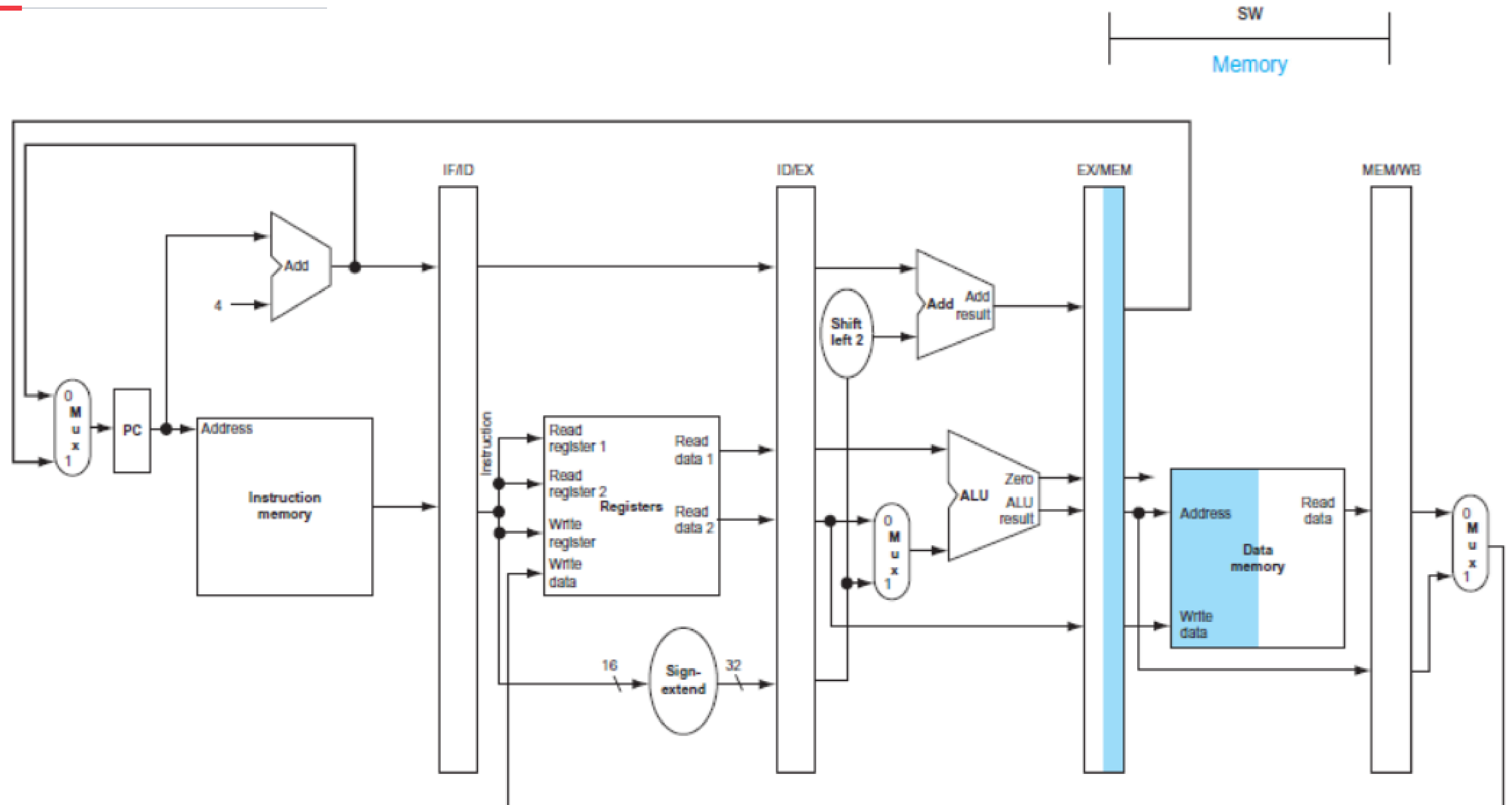




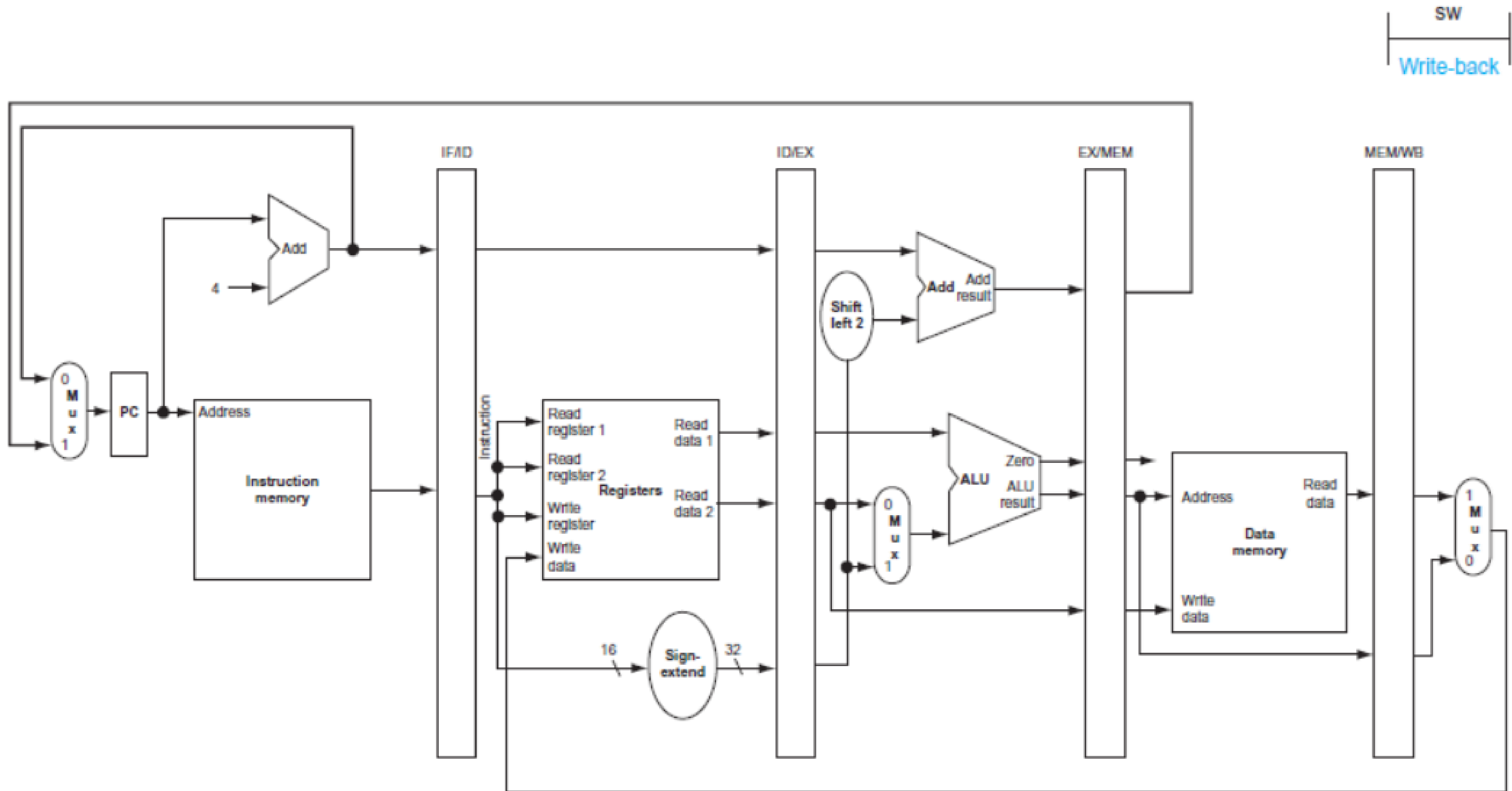
# Sw Example - Pipelined Datapath



# Sw Example - Pipelined Datapath

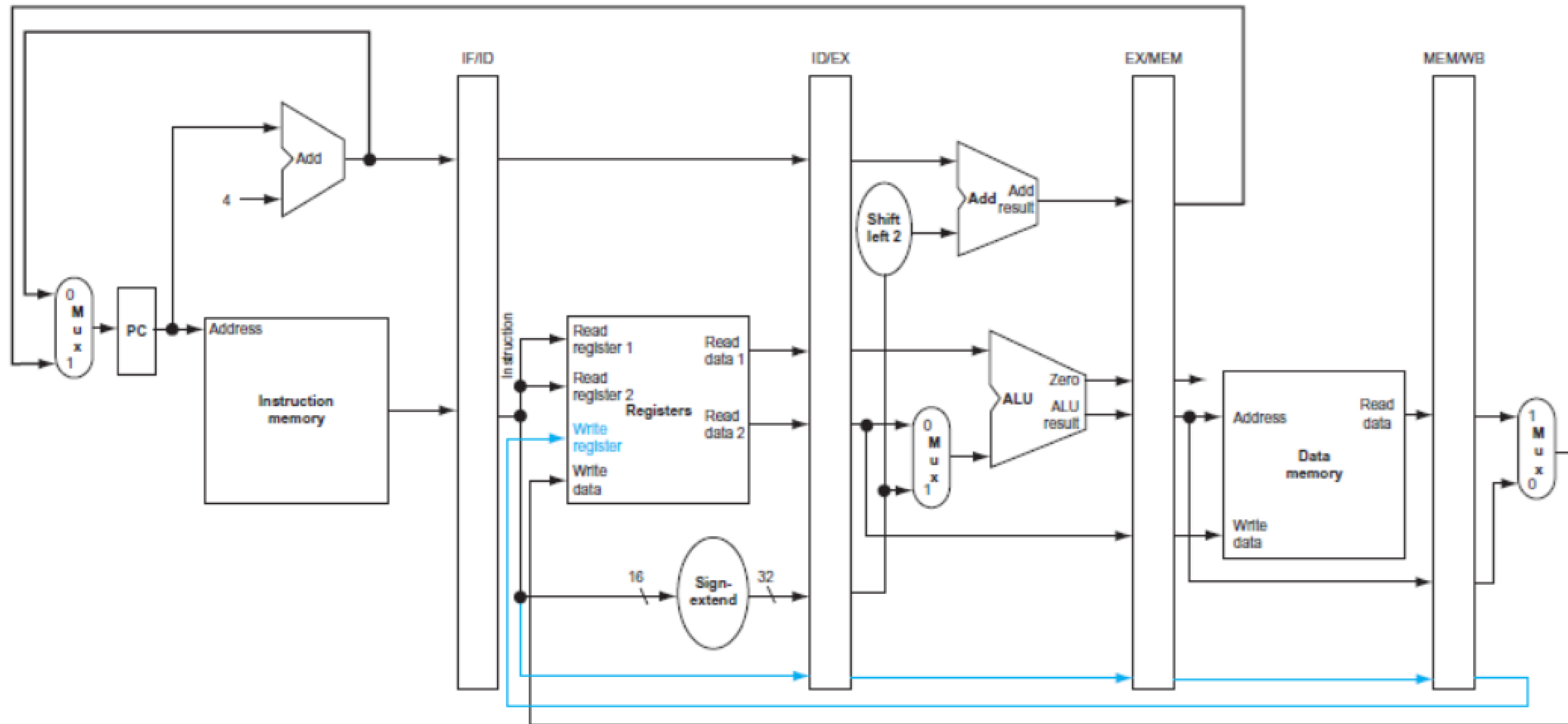


# Sw Example - Pipelined Datapath



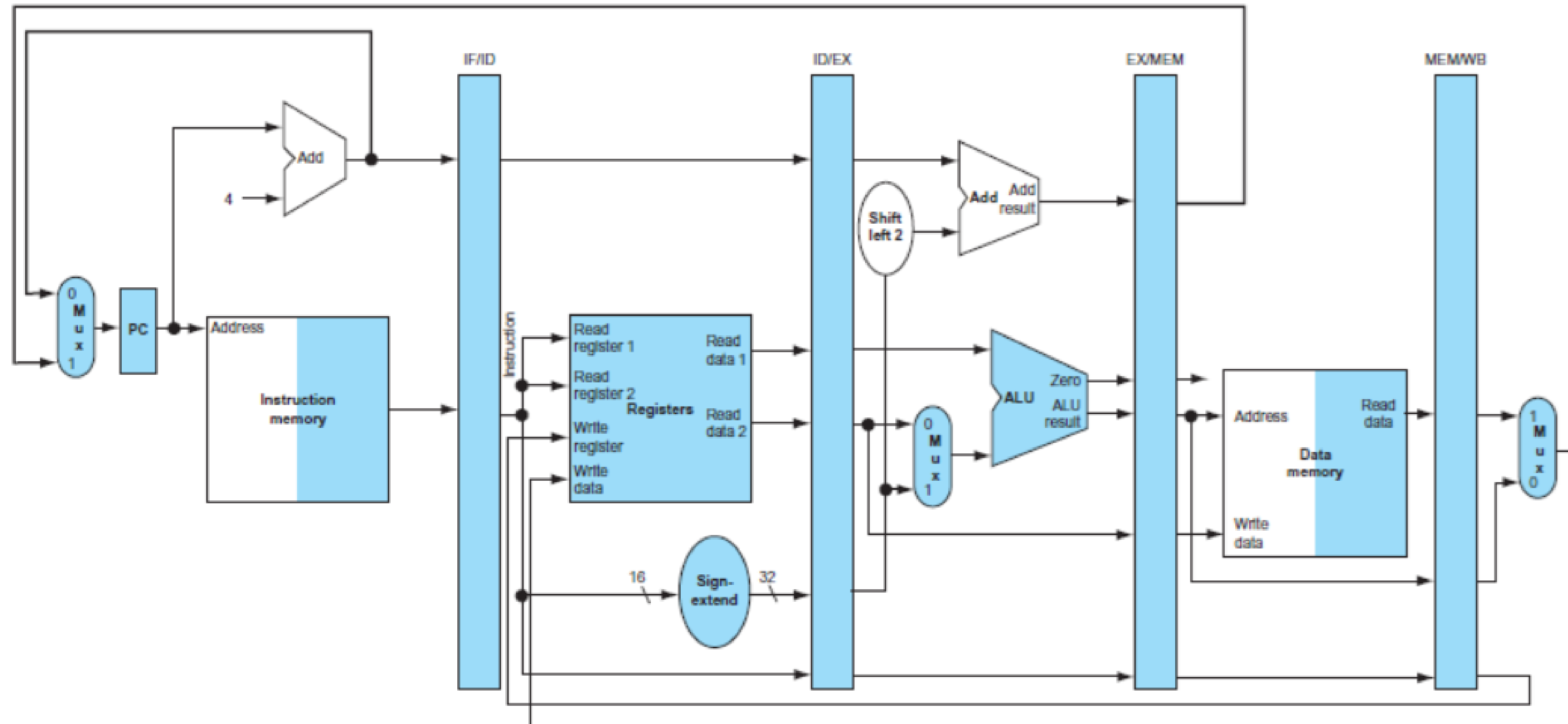


# Pipelined Datapath- Corrected Version



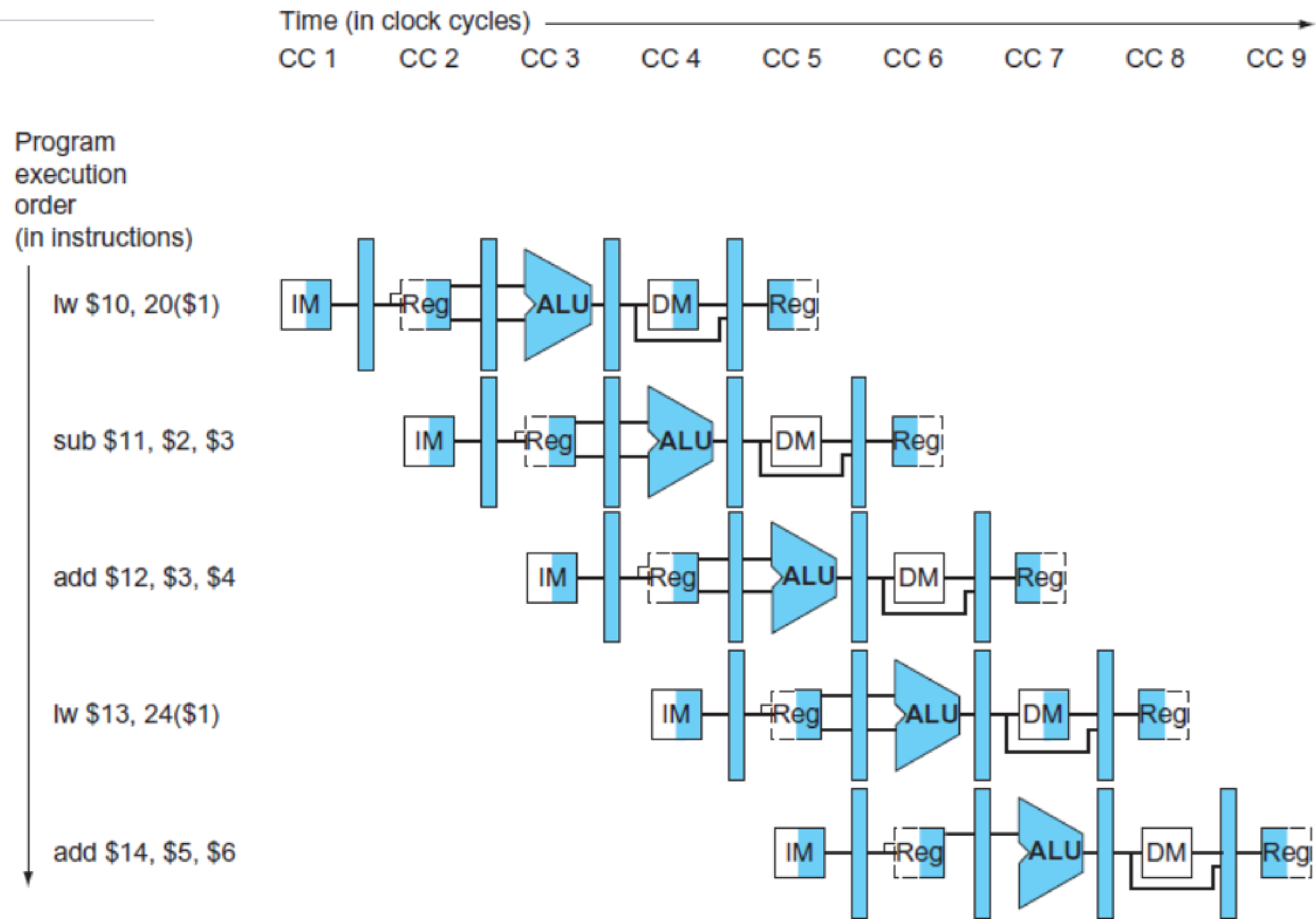
**FIGURE 4.41** The corrected pipelined datapath to handle the load instruction properly. The write register number now comes from the MEM/WB pipeline register along with the data. The register number is passed from the ID pipe stage until it reaches the MEM/WB pipeline register, adding five more bits to the last three pipeline registers. This new path is shown in color.

# Lw Example - Pipelined Datapath



**FIGURE 4.42** The portion of the datapath in Figure 4.41 that is used in all five stages of a load instruction.

# Representations of Pipelined Datapath



**FIGURE 4.43** Multiple-clock-cycle pipeline diagram of five instructions. This style of pipeline representation shows the complete



# Representations of Pipelined Datapath

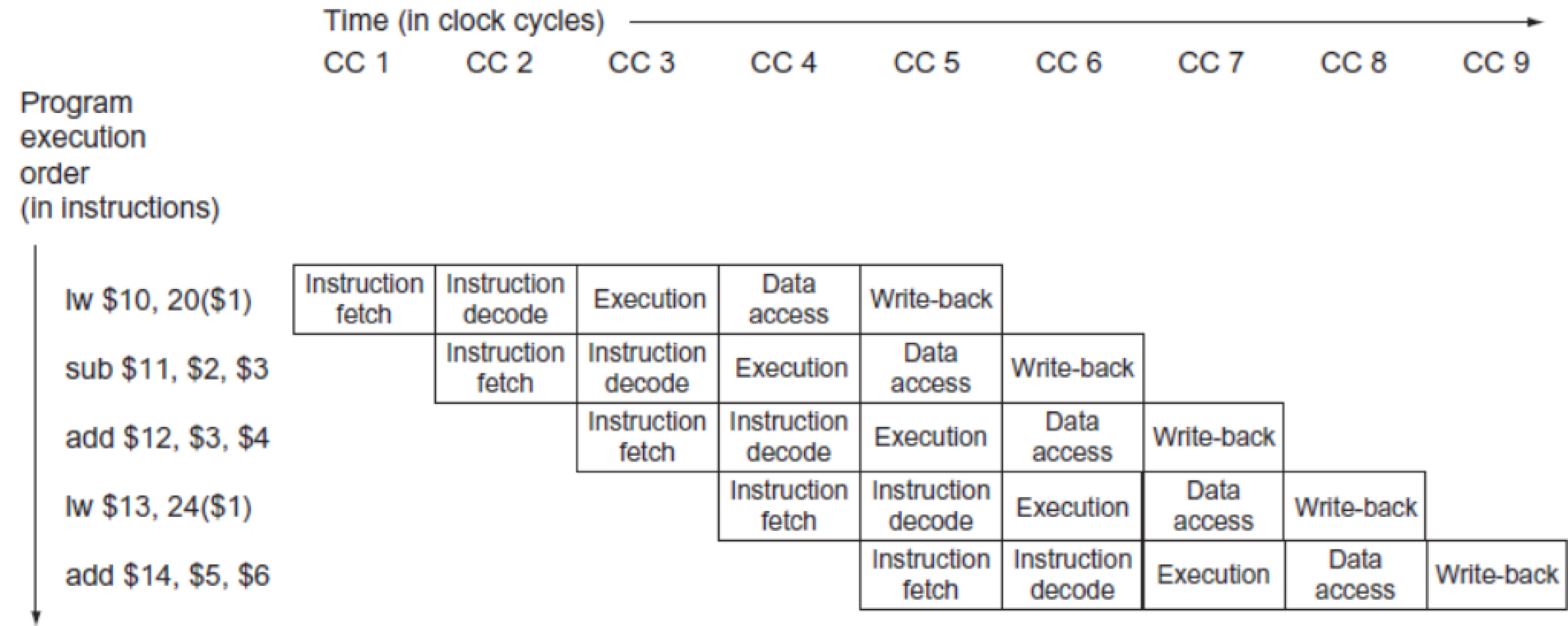
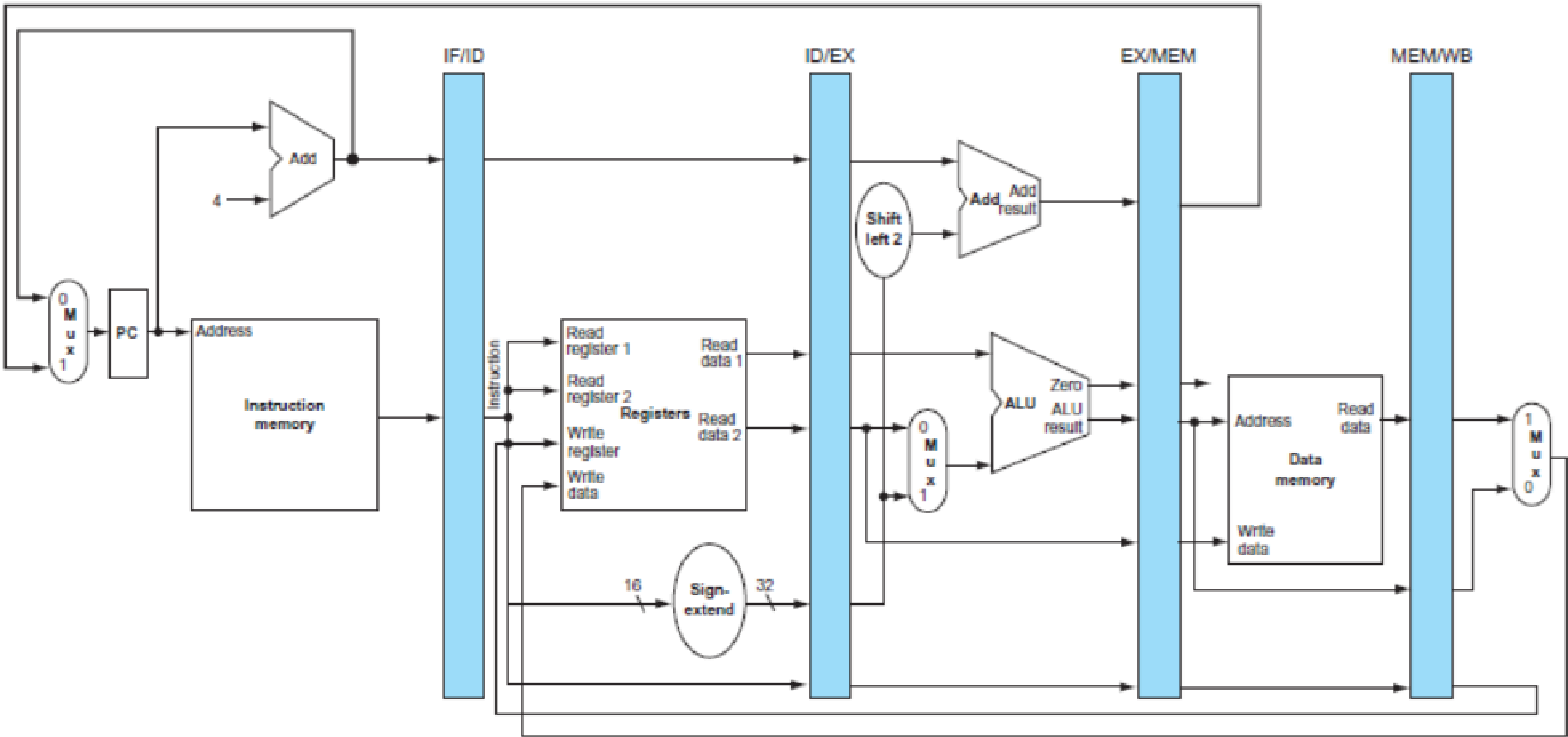


FIGURE 4.44 Traditional multiple-clock-cycle pipeline diagram of five instructions in Figure 4.43.



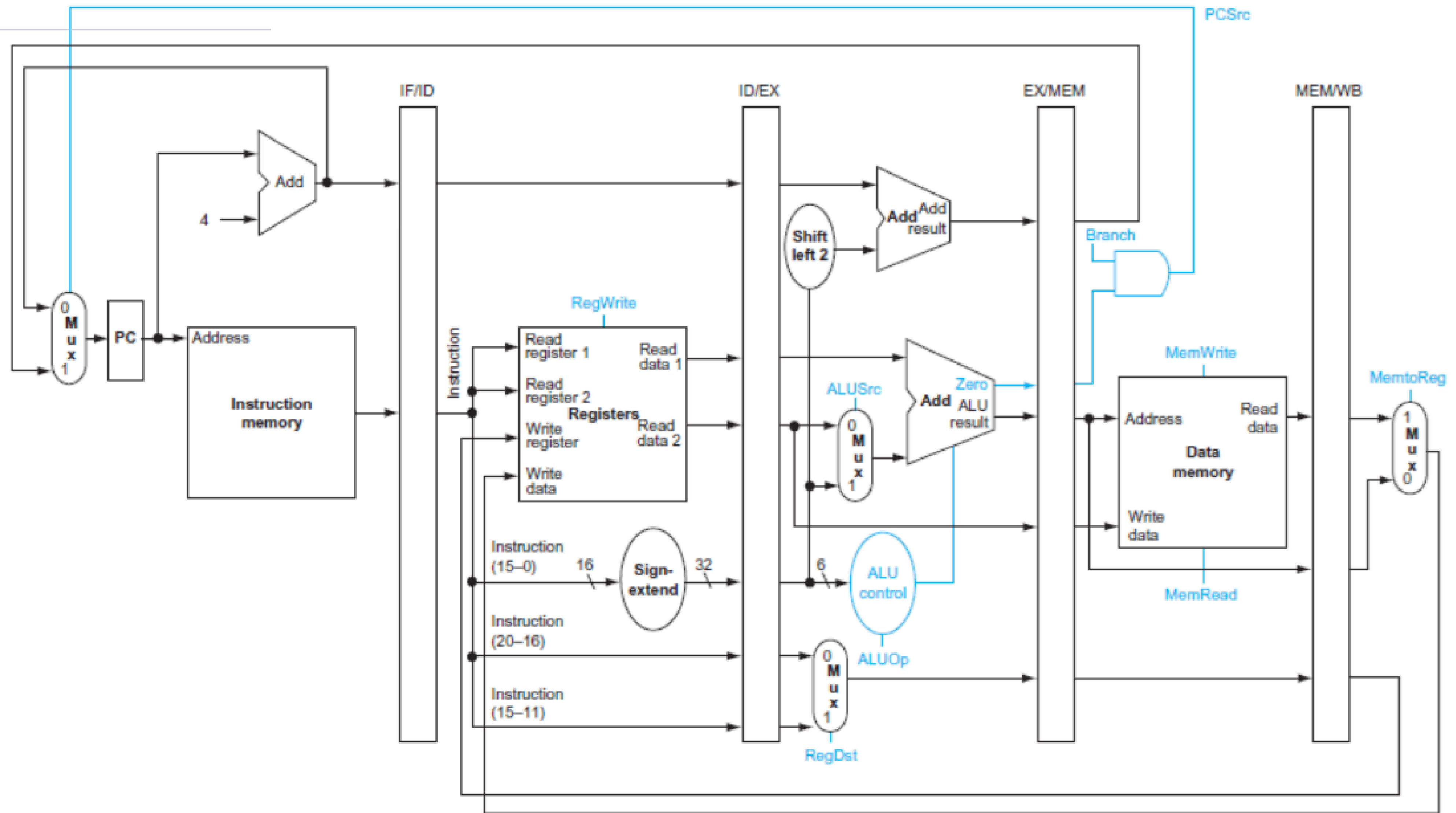
# Representations of Pipelined Datapath

|                    |                    |                    |                    |                  |
|--------------------|--------------------|--------------------|--------------------|------------------|
| add \$14, \$5, \$6 | lw \$13, 24 (\$1)  | add \$12, \$3, \$4 | sub \$11, \$2, \$3 | lw \$10, 20(\$1) |
| Instruction fetch  | Instruction decode | Execution          | Memory             | Write-back       |



**FIGURE 4.45** The single-clock-cycle diagram corresponding to **clock cycle 5 of the pipeline** in Figures 4.43 and 4.44. As you can see, a single-clock-cycle figure is a vertical slice through a multiple-clock-cycle diagram.

# Pipelined Control



**FIGURE 4.46** The pipelined datapath of Figure 4.41 with the control signals identified. This datapath borrows the control

# Pipelined Control

| Instruction opcode | ALUOp | Instruction operation | Function code | Desired ALU action | ALU control input |
|--------------------|-------|-----------------------|---------------|--------------------|-------------------|
| LW                 | 00    | load word             | XXXXXX        | add                | 0010              |
| SW                 | 00    | store word            | XXXXXX        | add                | 0010              |
| Branch equal       | 01    | branch equal          | XXXXXX        | subtract           | 0110              |
| R-type             | 10    | add                   | 100000        | add                | 0010              |
| R-type             | 10    | subtract              | 100010        | subtract           | 0110              |
| R-type             | 10    | AND                   | 100100        | AND                | 0000              |
| R-type             | 10    | OR                    | 100101        | OR                 | 0001              |
| R-type             | 10    | set on less than      | 101010        | set on less than   | 0111              |

**FIGURE 4.47** A copy of Figure 4.12. This figure shows how the ALU control bits are set depending on the ALUOp control bits and the different function codes for the R-type instruction.

| Instruction | Execution/address calculation stage control lines |        |        |        | Memory access stage control lines |          |           | Write-back stage control lines |           |
|-------------|---------------------------------------------------|--------|--------|--------|-----------------------------------|----------|-----------|--------------------------------|-----------|
|             | RegDst                                            | ALUOp1 | ALUOp0 | ALUSrc | Branch                            | Mem-Read | Mem-Write | Reg-Write                      | Memto-Reg |
| R-format    | 1                                                 | 1      | 0      | 0      | 0                                 | 0        | 0         | 1                              | 0         |
| lw          | 0                                                 | 0      | 0      | 1      | 0                                 | 1        | 0         | 1                              | 1         |
| sw          | X                                                 | 0      | 0      | 1      | 0                                 | 0        | 1         | 0                              | X         |
| beq         | X                                                 | 0      | 1      | 0      | 1                                 | 0        | 0         | 0                              | X         |

**FIGURE 4.49** The values of the control lines are the same as in Figure 4.18, but they have been shuffled into three groups corresponding to the last three pipeline stages.



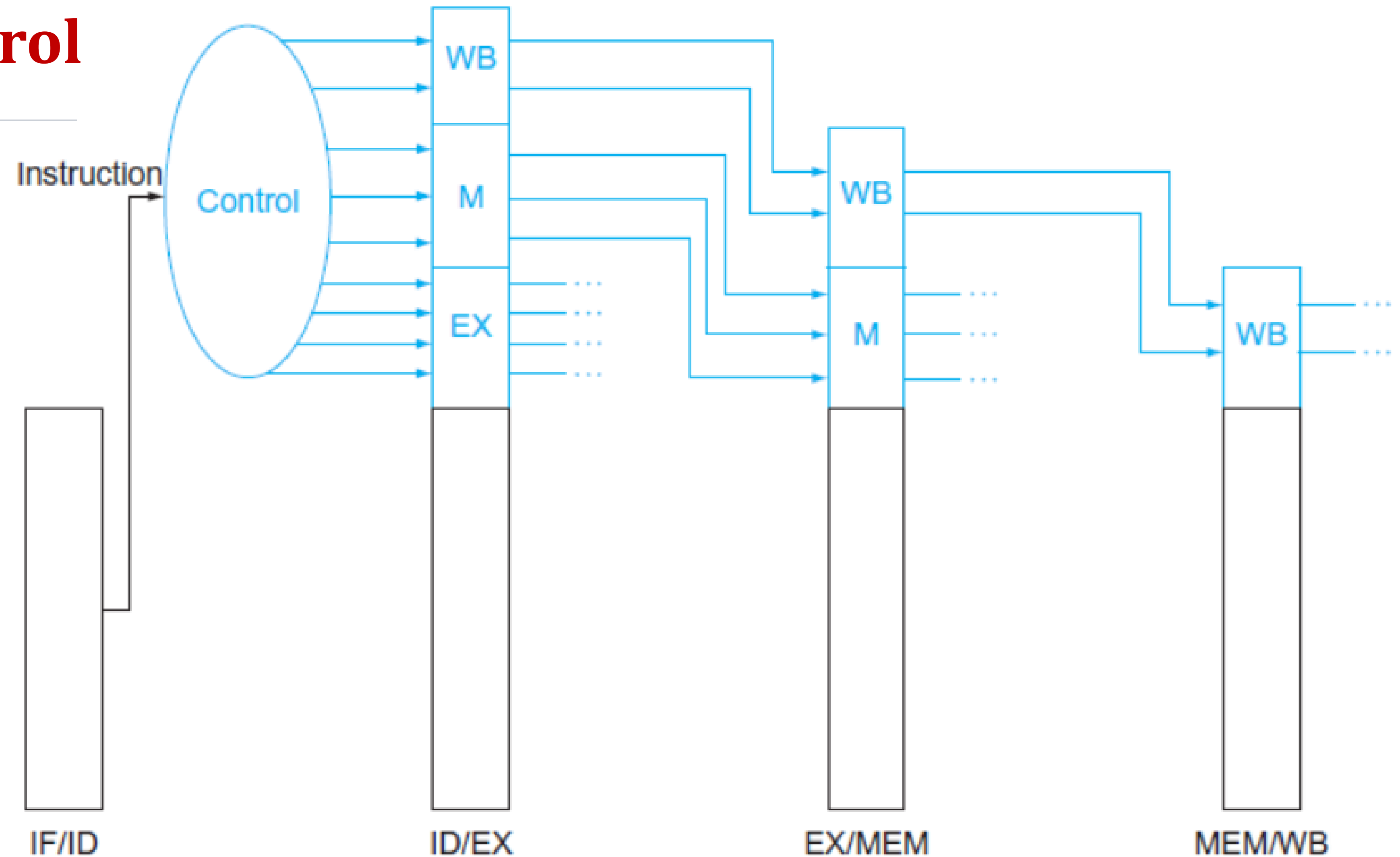
# Pipelined Control

| Signal name | Effect when deasserted (0)                                                                   | Effect when asserted (1)                                                                                |
|-------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| RegDst      | The register destination number for the Write register comes from the rt field (bits 20:16). | The register destination number for the Write register comes from the rd field (bits 15:11).            |
| RegWrite    | None.                                                                                        | The register on the Write register input is written with the value on the Write data input.             |
| ALUSrc      | The second ALU operand comes from the second register file output (Read data 2).             | The second ALU operand is the sign-extended, lower 16 bits of the instruction.                          |
| PCSrc       | The PC is replaced by the output of the adder that computes the value of PC + 4.             | The PC is replaced by the output of the adder that computes the branch target.                          |
| MemRead     | None.                                                                                        | Data memory contents designated by the address input are put on the Read data output.                   |
| MemWrite    | None.                                                                                        | Data memory contents designated by the address input are replaced by the value on the Write data input. |
| MemtoReg    | The value fed to the register Write data input comes from the ALU.                           | The value fed to the register Write data input comes from the data memory.                              |

**FIGURE 4.48 A copy of Figure 4.16.** The function of each of seven control signals is defined. The ALU control lines (ALUOp) are defined in the second column of Figure 4.47. When a 1-bit control to a 2-way multiplexor is asserted, the multiplexor selects the input corresponding to 1. Otherwise, if the control is deasserted, the multiplexor selects the 0 input. Note that PCSrc is controlled by an AND gate in Figure 4.46. If the Branch signal and the ALU Zero signal are both set, then PCSrc is 1; otherwise, it is 0. Control sets the Branch signal only during a beq instruction; otherwise, PCSrc is set to 0.



# Pipelined Control



**FIGURE 4.50 The control lines for the final three stages.** Note that four of the nine control lines are used in the EX phase, with the remaining five control lines passed on to the EX/MEM pipeline register extended to hold the control lines; three are used during the MEM stage, and the last two are passed to MEM/WB for use in the WB stage.